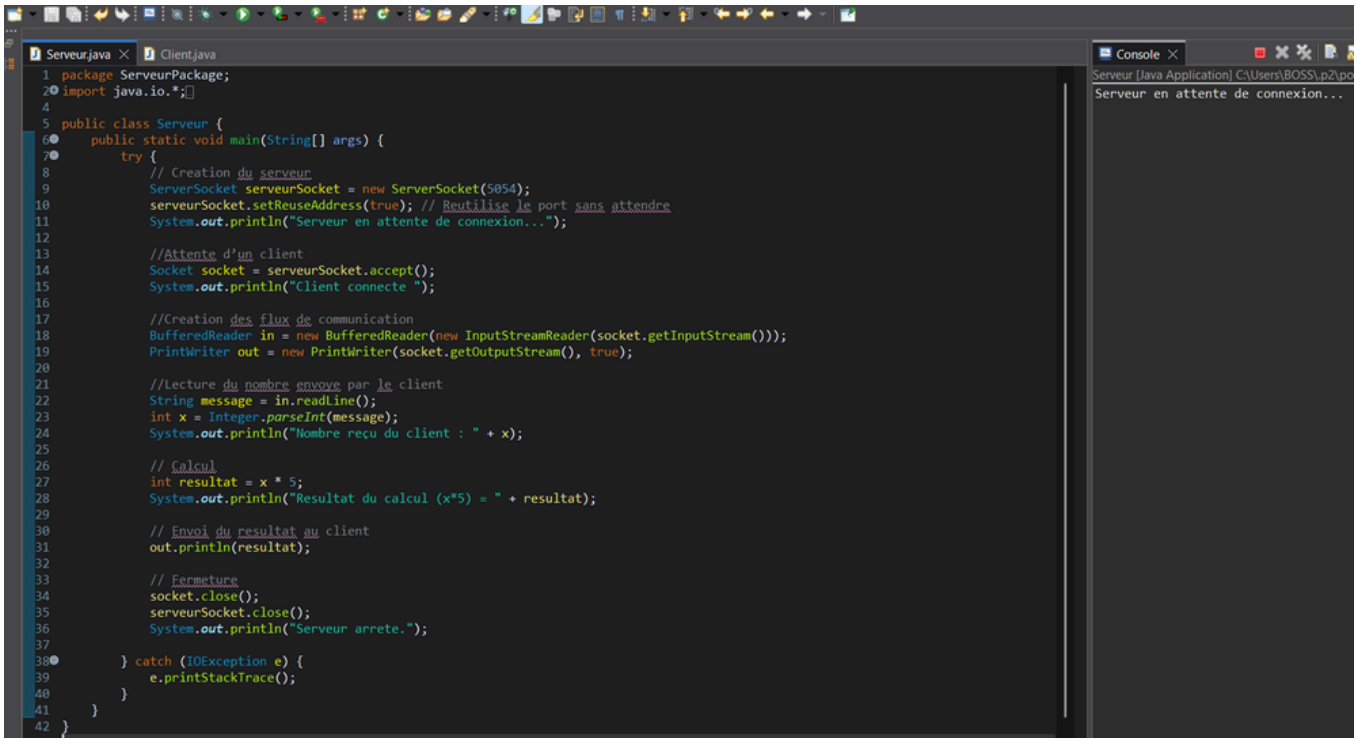
# COMPTE RENDU TP1 A1.1

**EYA CHIKHAOUI**

3e Licence Sciences de l'Informatique (LSI 3 – 1.2)

## Introduction

Ce premier TP du module Développement d'applications réparties a pour objectif de familiariser l'étudiant avec la communication entre processus via le protocole TCP en utilisant les sockets du langage Java.
Le travail consiste à étendre une application Client/Serveur de base afin de réaliser un service de calculatrice.
Dans ce service, le client saisit un entier x et l'envoie au serveur. Ce dernier reçoit la valeur, effectue le calcul x * 5, puis renvoie le résultat au client.
Cette activité permet de comprendre le modèle Client/Serveur, les étapes d'une communication réseau fiable et la gestion des flux d'entrée/sortie en Java.




```java
package ServeurPackage;
import java.io.*;

public class Serveur {
    public static void main(String[] args) {
        try {
            // Creation du serveur
            ServerSocket serveurSocket = new ServerSocket(5054);
            serveurSocket.setReuseAddress(true); // Reutilise le port sans attendre
            System.out.println("Serveur en attente de connexion...");

            //Attente d'un client
            Socket socket = serveurSocket.accept();
            System.out.println("Client connecte ");

            //Creation des flux de communication
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

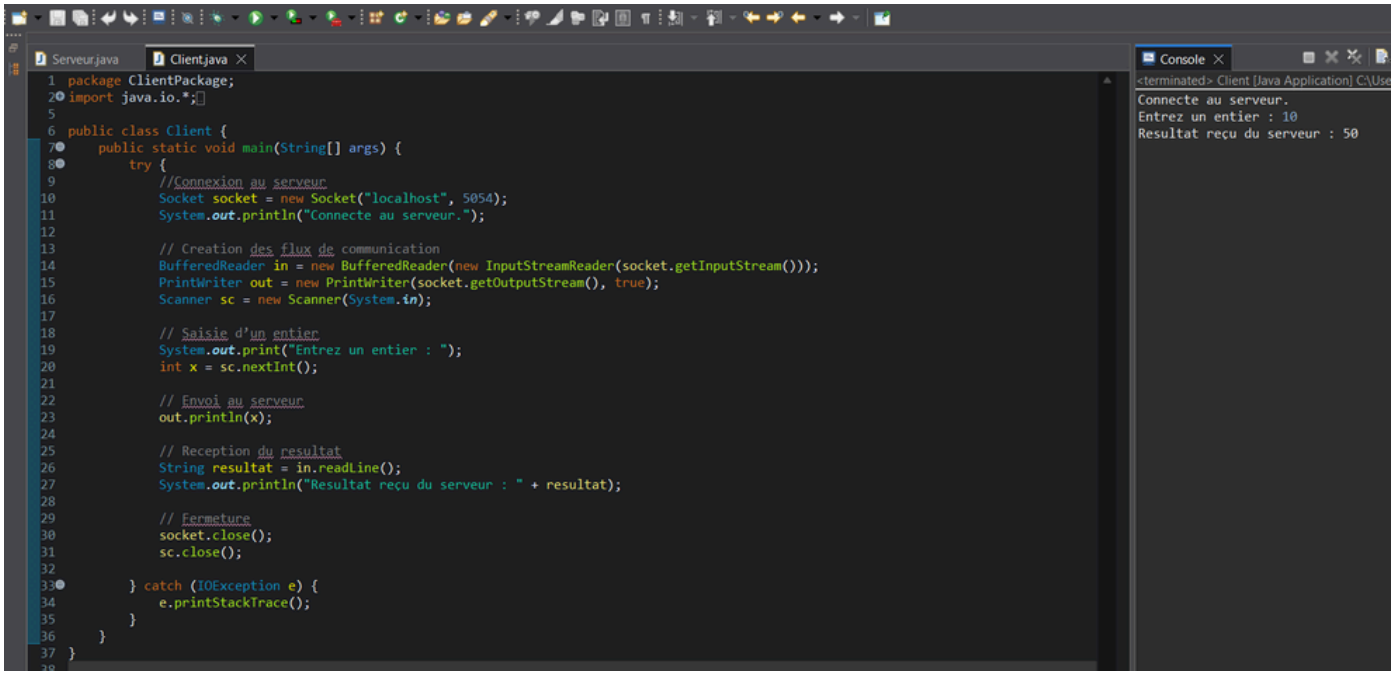
            //Lecture du nombre envoye par le client
            String message = in.readLine();
            int x = Integer.parseInt(message);
            System.out.println("Nombre reçu du client : " + x);

            // Calcul
            int resultat = x * 5;
            System.out.println("Resultat du calcul (x*5) = " + resultat);

            // Envoi du resultat au client
            out.println(resultat);

            // Fermeture
            socket.close();
            serveurSocket.close();
            System.out.println("Serveur arrete.");

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```


Serveur.java
Client.java
1 package ClientPackage;
2 import java.io.*;
5
6 public class Client {
7 public static void main(String[] args) {
8 try {
9 //Connexion au serveur
10 Socket socket = new Socket("localhost", 5054);
11 System.out.println("Connecte au serveur.");
12
13 // Creation des flux de communication
14 BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
15 PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
16 Scanner sc = new Scanner(System.in);
17
18 // Saisie d'un entier
19 System.out.print("Entrez un entier : ");
20 int x = sc.nextInt();
21
22 // Envoi au serveur
23 out.println(x);
24
25 // Reception du resultat
26 String resultat = in.readLine();
27 System.out.println("Resultat reçu du serveur : " + resultat);
28
29 // Fermeture
30 socket.close();
31 sc.close();
32
33 } catch (IOException e) {
34 e.printStackTrace();
35 }
36 }
37 }
Console
<terminated> Client [Java Application] C:\Use
Connecte au serveur.
Entrez un entier : 10
Resultat reçu du serveur : 50

## Interprétation des résultats :

Cette activité a permis de comprendre les bases de la communication par sockets :

- Le serveur reste bloqué en attente jusqu'à l'arrivée d'un client.
- Le client initie la connexion à travers un port TCP défini.
- La synchronisation entre les deux processus démontre le modèle de communication connecté et fiable offert par TCP.

***lien de tp dans github:*** https://github.com/eyaChikhaoui/TP1-DAR2026.git

**Conclusion :**
Ce premier TP m'a permis de maîtriser les fondements du modèle Client/Serveur à travers l'API Socket de Java.
Les étapes de configuration, d'exécution et de mise en ligne sur GitHub m'ont également permis d'acquérir des compétences pratiques en développement collaboratif et en gestion de version.
Cette base sera essentielle pour les prochaines activités de développement réparti, où la communication et l'échange de données deviendront plus complexes.

Je reste disponible pour toute remarque ou suggestion d'amélioration.
**Merci pour votre attention.**